

Lab 06: Design Pattern

Scenario 1: RPG Game

Concept: Decorator Pattern for enhancing character attack behavior:

- Attack – The common interface/supertype for both the base attack and all decorators to ensure interchangeability.
- BaseAttack – The concrete component containing core damage logic (damage + weapon damage).
- AttackDecorator – The abstract decorator that wraps an Attack object and delegates behavior.
- WarriorStyleDecorator – Decorator moving specialized 1.5x damage logic from the Warrior class.
- MageStyleDecorator – Decorator moving mana-based spell bonus logic from the Mage class.
- ArcherStyleDecorator – Decorator moving hit/miss and range bonus logic from the Archer class.

Instructions:

1. **Reuse classes from Lab 4 Scenario 2** as the foundation for refactoring.
2. **Add the Attack Component Interface (Attack.java)** with a method signature: void attack(Character attacker, Destructible target).
3. **Implement the Base Concrete Component (BaseAttack.java)** to handle the core "Raw Attack Damage" calculation previously found in Character.attack().
4. **Create the Abstract Decorator (AttackDecorator.java)** which stores a protected final Attack wrappedAttack.
5. **Refactor the Character class to use Composition:**
 - Add an Attack attribute.
 - Initialize the attribute to new BaseAttack() in the constructor.
 - Replace the body of attack(Destructible target) to delegate to attack.attack(this, target).
6. **Move Class-Specific Logic into Style Decorators:**
 - Create WarriorStyleDecorator, MageStyleDecorator, and ArcherStyleDecorator.
 - In the constructors of Warrior, Mage, and Archer, use setAttack() to wrap the existing attack behavior with their respective Style Decorator (e.g., setAttack(new WarriorStyleDecorator(getAttack()))).
 - Remove the attack() method in all Warrior, Mage, and Archer.

7. Add Reusable Generic Effect Decorators:

- Implement CriticalStrikeDecorator, DamageBoostDecorator, and LifeStealDecorator.
- These should stack logic before or after calling wrappedAttack.attack().

8. In RPGGameApp, demonstrate runtime composition by creating a chain for the warrior.

```
/**
 * Component interface for decorator-based attack processing.
 */
public interface Attack {
    void attack(Character attacker, Destructible target);
}
```

```
/**
 * Concrete base component that performs a regular attack.
 */
public class BaseAttack implements Attack {

    @Override
    public void attack(Character attacker, Destructible target) {
        if (!attacker.isAlive()) {
            System.out.println(attacker.getName() + " cannot attack because they
have fainted.");
            return;
        }

        int totalDamage = attacker.getDamage() +
attacker.getWeapon().getBaseDamage();
        System.out.println(" ⚔️ Raw Attack Damage: " + totalDamage);
        target.takeDamage(totalDamage);
    }
}
```

```
/**
 * Abstract decorator that wraps another Attack component.
 */
public abstract class AttackDecorator implements Attack {

    protected final Attack wrappedAttack;

    protected AttackDecorator(Attack wrappedAttack) {
        if (wrappedAttack == null) {
            throw new IllegalArgumentException("Wrapped attack cannot be null.");
        }
        this.wrappedAttack = wrappedAttack;
    }
}
```

```

/**
 * Decorator for warrior-specific power strike behavior.
 */
public class WarriorStyleDecorator extends AttackDecorator {

    public WarriorStyleDecorator(Attack wrappedAttack) {
        super(wrappedAttack);
    }

    @Override
    public void attack(Character attacker, Destructible target) {
        int originalDamage = attacker.getDamage();
        int enhancedDamage = (int) Math.round(originalDamage * 1.5);
        String targetName = (target instanceof Character) ? ((Character)
target).getName() : "Target";

        attacker.setDamage(enhancedDamage);
        System.out.println("\n⚔️ " + attacker.getName() + " (Warrior) performs a
POWERFUL STRIKE on " + targetName + "!");
        System.out.println("    Base Damage: " + originalDamage + " -> Enhanced: "
+ enhancedDamage + " (1.5x multiplier)");

        try {
            wrappedAttack.attack(attacker, target);
        } finally {
            attacker.setDamage(originalDamage);
        }
    }
}

```

```

/**
 * Decorator for mage spellcasting behavior.
 */
public class MageStyleDecorator extends AttackDecorator {

    private static final int MAGIC_MISSILE_COST = 30;

    public MageStyleDecorator(Attack wrappedAttack) {
        super(wrappedAttack);
    }

    @Override
    public void attack(Character attacker, Destructible target) {
        if (!(attacker instanceof Mage)) {
            wrappedAttack.attack(attacker, target);
            return;
        }

        Mage mage = (Mage) attacker;
        String targetName = (target instanceof Character) ? ((Character)
target).getName() : "Target";

        if (mage.getMana() < MAGIC_MISSILE_COST) {

```

```

        System.out.println("\n💎 " + mage.getName() + " attempts to cast Magic Missile, but has insufficient mana!");
        System.out.println(
            "    Current Mana: " + mage.getMana() + "/" + mage.getMaxMana() + " (Need: " + MAGIC_MISSILE_COST + ")");
        return;
    }

    int originalDamage = mage.getDamage();
    mage.setDamage(originalDamage + 10);
    mage.setMana(mage.getMana() - MAGIC_MISSILE_COST);

    System.out.println("\n💎 " + mage.getName() + " (Mage) casts MAGIC MISSILE at " + targetName + "!");
    System.out.println("    Spell Bonus: +10 | Mana Used: " + MAGIC_MISSILE_COST + " | Remaining: " + mage.getMana() + "/" + mage.getMaxMana());

    try {
        wrappedAttack.attack(attacker, target);
    } finally {
        mage.setDamage(originalDamage);
    }
}
}

```

```

/**
 * Decorator for archer precision and range advantage behavior.
 */
public class ArcherStyleDecorator extends AttackDecorator {

    public ArcherStyleDecorator(Attack wrappedAttack) {
        super(wrappedAttack);
    }

    @Override
    public void attack(Character attacker, Destructible target) {
        if (!(attacker instanceof Archer)) {
            wrappedAttack.attack(attacker, target);
            return;
        }

        Archer archer = (Archer) attacker;
        String targetName = (target instanceof Character) ? ((Character) target).getName() : "Target";
        boolean hitSuccessful = (Math.random() * 100) < archer.getAccuracy();

        if (!hitSuccessful) {
            System.out.println("\n🏹 " + archer.getName() + " (Archer) shoots at " + targetName + "...");
            System.out.println("    MISS! The arrow flies wide! (Accuracy: " + archer.getAccuracy() + "%)");
            return;
        }
    }
}

```

```

        int originalDamage = archer.getDamage();
        int bonusDamage = archer.hasRangeAdvantage() ? (int)
Math.round(originalDamage * 0.3) : 0;
        int enhancedDamage = originalDamage + bonusDamage;
        archer.setDamage(enhancedDamage);

        System.out.println("\n🏹 " + archer.getName() + " (Archer) shoots a
precise arrow at " + targetName + "!");
        System.out.println("    Base Damage: " + originalDamage + " + Range Bonus:
" + bonusDamage + " = " + enhancedDamage);
        System.out.println("    Accuracy: " + archer.getAccuracy() + "% (HIT!)");

        try {
            wrappedAttack.attack(attacker, target);
        } finally {
            archer.setDamage(originalDamage);
        }
    }
}

```

```

/**
 * Generic decorator that applies critical-strike chance.
 */
public class CriticalStrikeDecorator extends AttackDecorator {

    public CriticalStrikeDecorator(Attack wrappedAttack) {
        super(wrappedAttack);
    }

    @Override
    public void attack(Character attacker, Destructible target) {
        int originalDamage = attacker.getDamage();
        boolean critical = (Math.random() * 100) < 25;

        if (critical) {
            attacker.setDamage(originalDamage * 2);
            String targetName = (target instanceof Character) ? ((Character)
target).getName() : "Target";
            System.out.println("\n★ " + attacker.getName() + " lands a CRITICAL
STRIKE on " + targetName + "!");
            System.out.println("    Base Damage: " + originalDamage + " -> Critical: " +
attacker.getDamage() + " (2x)");
        }

        try {
            wrappedAttack.attack(attacker, target);
        } finally {
            attacker.setDamage(originalDamage);
        }
    }
}

```

```

    }
}
}

```

```

/**
 * Generic decorator that adds a flat damage boost.
 */
public class DamageBoostDecorator extends AttackDecorator {

    private final int bonusDamage;

    public DamageBoostDecorator(Attack wrappedAttack, int bonusDamage) {
        super(wrappedAttack);
        if (bonusDamage < 0) {
            throw new IllegalArgumentException("Bonus damage cannot be negative.");
        }
        this.bonusDamage = bonusDamage;
    }

    @Override
    public void attack(Character attacker, Destructible target) {
        int originalDamage = attacker.getDamage();
        attacker.setDamage(originalDamage + bonusDamage);

        try {
            System.out.println(" [Decorator] Damage Boost activated: +" + bonusDamage
+ " damage");
            wrappedAttack.attack(attacker, target);
        } finally {
            attacker.setDamage(originalDamage);
        }
    }
}

```

```

/**
 * Generic decorator that restores HP after an attack.
 */
public class LifeStealDecorator extends AttackDecorator {

    private final double ratio;

    public LifeStealDecorator(Attack wrappedAttack, double ratio) {
        super(wrappedAttack);
    }
}

```

```

        if (ratio < 0 || ratio > 1) {
            throw new IllegalArgumentException("Life steal ratio must be between 0 and
1.");
        }
        this.ratio = ratio;
    }

    @Override
    public void attack(Character attacker, Destructible target) {
        int hpBefore = attacker.getHealthPoints();

        wrappedAttack.attack(attacker, target);

        if (!attacker.isAlive()) {
            return;
        }

        int healBase = attacker.getDamage() + attacker.getWeapon().getBaseDamage();
        int healAmount = Math.max(1, (int) Math.round(healBase * ratio));
        int missing = attacker.getMaxHealthPoints() - attacker.getHealthPoints();

        if (missing <= 0) {
            return;
        }

        int restored = Math.min(healAmount, missing);
        attacker.setHealthPoints(attacker.getHealthPoints() + restored);

        System.out.println(" [Decorator] Life Steal restored " + restored + " HP to
" + attacker.getName()
            + " (" + hpBefore + " -> " + attacker.getHealthPoints() + ")");
    }
}

```

RPGGameApp

```

public class RPGGameApp {

    public static void main(String[] args) {
        System.out.println("\n" + "=".repeat(70));
        System.out.println(" SCENARIO 4: INTERFACES - DESTRUCTIBLE &
CONSUMABLE");
        System.out.println("=".repeat(70));

        // ===== CREATE WEAPONS =====
        System.out.println("\n[STEP 1] Creating Weapons...");
        Weapon swordWeapon = new Weapon("Excalibur", "Melee", 30, "Slash

```

```

Attack");
    Weapon staffWeapon = new Weapon("Staff of Mysticism", "Magic", 20, "Spell
Cast");
    Weapon bowWeapon = new Weapon("Bow of Elven Make", "Ranged", 18, "Precise
Shot");

    // ===== CREATE CHARACTERS =====
    System.out.println("\n[STEP 2] Creating Characters...");
    Warrior warrior = new Warrior("Aragorn", 12, 2000, 25, 8, 15,
swordWeapon);
    Mage mage = new Mage("Gandalf", 14, 1200, 20, 5, 150, staffWeapon);
    Archer archer = new Archer("Legolas", 13, 1400, 22, 6, 85, bowWeapon);

    // ===== CREATE DESTRUCTIBLE OBJECTS (NON-CHARACTER)
    System.out.println("\n[STEP 3] Creating Destructible Objects (Non-
Character)...");
    WoodenBox treasureBox = new WoodenBox("Treasure Chest", 300, true, "1000
Gold Coins");
    WoodenBox crate = new WoodenBox("Supply Crate", 150, false, "Medicine &
Supplies");

    // ===== CREATE CONSUMABLE ITEMS =====
    System.out.println("\n[STEP 4] Creating Consumable Items...");
    HealthPotion potionGreater = new HealthPotion("Greater Health Potion",
100, 3);
    HealthPotion potionMinor = new HealthPotion("Minor Health Potion", 10,
10);

    // ===== DISPLAY INITIAL STATUS =====
    System.out.println("\n[STEP 5] Initial Status:");
    System.out.println("\n--- CHARACTERS (Destructible) ---");
    warrior.displayCharacterDetails();
    mage.displayCharacterDetails();
    archer.displayCharacterDetails();

    System.out.println("\n--- DESTRUCTIBLE OBJECTS (Non-Character) ---");
    treasureBox.displayBoxDetails();
    crate.displayBoxDetails();

    System.out.println("\n--- CONSUMABLE ITEMS ---");
    potionGreater.displayPotionDetails();
    potionMinor.displayPotionDetails();

    // ===== DEMONSTRATE INTERFACE POLYMORPHISM: DESTRUCTIBLE
    // =====
    System.out.println("\n" + "=".repeat(70));
    System.out.println(" [STEP 6] INTERFACE POLYMORPHISM: Destructible
Objects");
    System.out.println("=".repeat(70));
    System.out.println("\n💎 KEY CONCEPT: Warriors can attack BOTH Characters
and Objects!");
    System.out.println(" Both implement Destructible, but DON'T share
inheritance.\n");

    // Create a list of destructible items (both Characters and non-
Characters!)

```



```

List<Destructible> battleTargets = new ArrayList<>();
battleTargets.add(mage); // Character is Destructible
battleTargets.add(warrior); // Another Character is Destructible
battleTargets.add(treasureBox); // WoodenBox is Destructible (NOT a
Character!)
battleTargets.add(crate); // Another WoodenBox

System.out.println("--- Aragorn and Mage attacks each other (Character -
Destructible) ---");
warrior.setAttack(
    new LifeStealDecorator(
        new DamageBoostDecorator(
            new CriticalStrikeDecorator(warrior.getAttack()),
            8),
        0.15));
System.out.println("    Decorator Chain Applied: WarriorStyle ->
CriticalStrike -> DamageBoost -> LifeSteal");
for (int i = 0; i < 5; i++) {
    warrior.attack(mage);
    mage.attack(warrior);
}

System.out.println("\n--- Aragorn continues attacking the Treasure Chest
---");
while (!treasureBox.isDestroyed()) {
    warrior.attack(treasureBox);
}

System.out.println("\n--- Legolas attacks a Supply Crate (Object -
Destructible) ---");
archer.attack(crate);

// ===== SHOW STATUS AFTER ATTACKS =====
System.out.println("\n[STEP 7] Status After Combat:");
System.out.println("\n--- Mage Status ---");
mage.displayCharacterDetails();

System.out.println("\n--- Destructible Objects Status ---");
treasureBox.displayBoxDetails();
crate.displayBoxDetails();

// ===== DEMONSTRATE INTERFACE POLYMORPHISM: CONSUMABLE =====
// =====
System.out.println("\n" + "=".repeat(70));
System.out.println("    [STEP 8] INTERFACE POLYMORPHISM: Consumable
Items");
System.out.println("=".repeat(70));
System.out.println("\n💎 KEY CONCEPT: Different consumable items, same
'use' method!\n");

// Create a list of consumable items
List<Consumable> inventory = new ArrayList<>();
inventory.add(potionGreater);
inventory.add(potionMinor);

System.out.println("--- Gandalf uses a Greater Health Potion ---");

```

```

inventory.get(0).use(mage);

System.out.println("\n--- Gandalf uses a Minor Health Potion ---");
inventory.get(1).use(mage);

// ===== OPEN DESTROYED BOX =====
System.out.println("\n[STEP 9] Breaking Open Boxes:");
System.out.println("\n--- Breaking open the Treasure Chest ---");
treasureBox.breakOpen();

System.out.println("\n--- Attempting to break the Supply Crate (still
intact) ---");
crate.breakOpen();

// ===== CROSS-HIERARCHY INTERACTION =====
System.out.println("\n" + "=".repeat(70));
System.out.println(" [STEP 10] CROSS-HIERARCHY INTERACTION: Stone Walls
& Castle Gates");
System.out.println("=".repeat(70));
System.out.println("\n💎 KEY CONCEPT: Warrior can attack NEW object types
WITHOUT recompilation!");
System.out.println(" StoneWall and CastleGate implement Destructible -
NO changes to Warrior!\n");

// Create new destructible structures
System.out.println("[Creating New Structures]");
StoneWall northWall = new StoneWall("North Wall", 500, "Outer Wall");
StoneWall eastWall = new StoneWall("East Wall", 400, "Inner Wall");
CastleGate mainGate = new CastleGate("Main Gate", 600, true, true);
CastleGate sideGate = new CastleGate("Side Gate", 300, false, false);

// Display initial status
System.out.println("\n--- Initial Structure Status ---");
northWall.displayWallDetails();
eastWall.displayWallDetails();
mainGate.displayGateDetails();
sideGate.displayGateDetails();

// Warrior attacks stone walls
System.out.println("\n--- Aragorn (Warrior) attacks the North Wall ---");
for (int i = 0; i < 8; i++) {
    warrior.attack(northWall);
    if (northWall.isDestroyed()) {
        break;
    }
}

// Archer attacks castle gate
System.out.println("\n--- Legolas (Archer) attacks the Main Gate
(Reinforced!) ---");
for (int i = 0; i < 10; i++) {
    archer.attack(mainGate);
    if (mainGate.isDestroyed()) {
        break;
    }
}

```

```

// Warrior attacks side gate
System.out.println("\n--- Aragorn (Warrior) attacks the Side Gate ---");
while (!sideGate.isDestroyed()) {
    warrior.attack(sideGate);
}

// Show final status
System.out.println("\n--- Final Structure Status ---");
northWall.displayWallDetails();
mainGate.displayGateDetails();
sideGate.displayGateDetails();

// Demonstrate polymorphism with mixed destructibles
System.out.println("\n" + "=".repeat(70));
System.out.println(" [STEP 11] POLYMORPHIC LIST: Mixed Destructible
Objects");
System.out.println("=".repeat(70));
System.out.println("\n💎 All objects in one list - Characters, Boxes,
Walls, Gates!\n");

List<Destructible> allDestructibles = new ArrayList<>();
allDestructibles.add(eastWall); // StoneWall
allDestructibles.add(crate); // WoodenBox
allDestructibles.add(archer); // Character

System.out.println("--- Mage attacking various targets from polymorphic
list ---");
for (Destructible target : allDestructibles) {
    if (target instanceof StoneWall) {
        System.out.println("\n>> Target: " + ((StoneWall)
target).getLocation());
    } else if (target instanceof WoodenBox) {
        System.out.println("\n>> Target: " + ((WoodenBox) target).getName());
    } else if (target instanceof Character) {
        System.out.println("\n>> Target: " + ((Character) target).getName());
    }
    mage.attack(target);
}
}
}

```